

CFD-ML-BASED TURBULENCE MODEL DEFINITION FOR FIDELITY IMPROVEMENT OF HORIZONTAL-AXIS WIND TURBINE RANS SIMULATIONS

**Andrea Carloni¹, Giovanni Delibra², Filippo De Girolamo³, Lorenzo Tieghi⁴,
Alessandro Corsini², Valerio Francesco Barnabei²**

1. Department of Electrical and Energy Engineering, Sapienza University of Rome, Italy;
email: andrea.carloni@uniroma1.it
2. Department of Mechanical and Aerospace Engineering, Sapienza University of Rome, Italy;
3. Division of Fluid Mechanics, KTH Royal Institute of Technology, Stockholm, Sweden;
4. Department of Civil, Environmental and Mechanical Engineering, University of Trento,
Trento, Italy;

ABSTRACT

Accurate prediction of horizontal-axis wind turbine (HAWT) wakes is essential for wind farm design, yet affordable tools struggle: Large Eddy Simulation (LES) is reliable but prohibitively expensive, while unsteady RANS (URANS) systematically mis-predicts the wake recovery. This work uses machine learning to define, rather than post-process, a machine learning-based turbulence closure of a $k-\omega$ SST RANS solver for HAWT wakes. From a single high-fidelity LES of the NREL-5MW turbine in a turbulent atmospheric boundary layer (ABL), an LES-equivalent eddy viscosity $\nu_{t,eq}$ is extracted as target; a K-means clustering segments the flow and a Multi-Layer Perceptron (MLP) learns the local correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,RANS}$. The machine learning models and correction is then embedded into a modified OpenFOAM PIMPLE solver. By design the closure is calibrated on this single case and transferred without retraining to unseen grids and inflow velocities, the core premise of the method. The network reproduces $\nu_{t,eq}$ with a validation $R^2 = 0.9952$; once embedded, the ML-based RANS model outperforms the standard $k-\omega$ SST model in near- and far-wake and, most notably, on ultra-coarse grids captures the post-breakdown recovery at a cost about 1.25 times lower than the reference LES, while also reproducing the rotor forces and power coefficients.

Keywords: Wind turbine wake, URANS, Machine learning, Turbulence modelling

1 INTRODUCTION

Wind energy is a cornerstone of the transition toward sustainable electricity generation. Turbines' rotor interactions with the upcoming wind flow generates low-speed, high turbulence wakes, which are characterised by multi-scale phenomena both in terms of space and time; downstream turbines operating within it suffer reduced power output and increased fatigue loads [1], so accurate wake prediction is essential for wind farm layout optimisation, energy-yield estimation and load assessment. Furthermore, wakes behaviour is deeply coupled with the atmospheric boundary layer (ABL), whose recovery is governed by shear-driven turbulent mixing [1], site topology and surface roughness. Available modelling strategies trade fidelity against cost: analytical wake models are near-instantaneous but strongly simplified; LES resolves the dominant structures but can demand 10^5 – 3×10^6 CPU hours per wind farm [2]; URANS is about two orders of magnitude cheaper [2], but its closure assumptions cause a systematic mis-prediction of the wake recovery [3].

Machine learning (ML) is increasingly used in wind energy, most often downstream of a physical model for power forecasting or layout optimisation, leaving the turbulence model unchanged [4]. A more recent line corrects or replaces the closure itself during the simulation [5, 6];

in the wind-energy context, corrective fields derived from time-averaged LES data have produced improved RANS wake predictions [2, 7].

This work contributes in that framework in three ways: a K-means clustering segments the domain into physically distinct regions [8, 9], moving the correction from a cell-wise to a region-wise basis that is more interpretable, less mesh-sensitive and embedd higher generalizability; a MLP learns the local cluster-based correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,RANS}$, embedded directly into the OpenFOAM PIMPLE solver so that it actively drives the momentum equations rather than being applied a posteriori. Crucially, the closure is deliberately trained on a single reference case and transferred without retraining to coarser grids and unseen inflow velocities: this single-case-to-many transfer is the design target of the method, not a data-scarcity workaround, and is shown to deliver LES-time-averaged-like wake representation at a cost lower than standard LES. In essence, ML is used here to define the turbulence model, within the scalar eddy-viscosity framework for direct solver embedding, rather than to post-process its output.

2 METHODOLOGY

The methodology combines a CFD part, used to generate the data and to test the model, and an ML part, used to train the clustering and neural-network models.

2.1 CFD dataset

Simulations are performed with OpenFOAM-v22.12 using an in-house Actuator Line Model (ALM) [10] based on the *turbinesFoam* library [11], under incompressible flow conditions, solving $\vec{U}$ and p with the PIMPLE algorithm. RANS simulations use the $k-\omega$ SST model [12]; LES uses the WALE model [13]. A logarithmic velocity profile is imposed at the inlet; at the LES inlet, synthetic turbulent fluctuations from the Kaimal spectrum [14] are superimposed following [15, 9].

To generate the regression data, a modified RANS run imports the mean $\vec{U}$ and p fields from the LES and, fixing those fields, performs a single step in which the ν_t field is solved consistently with the LES input inside the turbulence correction loop, yielding the modified eddy viscosity $\nu_{t,mod}$. The computational domain is a rectangular box with the rotor at $5D$ from the inlet, $15D$ from the outlet and 90 m from the terrain, extending $5D$ on each side and $8D$ in the vertical direction. The finest mesh has ~ 30.8 M cells (minimum cell size 1.75 m near the rotor). Coarser grids of ~ 15.9 , ~ 9.1 and ~ 4.2 M cells are obtained by varying the domain subdivision with same relative-to-max-cell-size refinement, and are used to assess the mesh-refinement influence on the trained models.

2.2 LES-equivalent eddy viscosity and ML target

Here ν_t denotes the kinematic turbulent viscosity entering the RANS equations through the effective viscosity $\nu_{eff} = \nu + \nu_t$. The reference LES equivalent eddy viscosity $\nu_{t,eq}$ is obtained by projecting the deviatoric part of the LES turbulent stress (sum of resolved and subgrid-scale contributions) onto the Boussinesq form, $\Sigma_{ij}^{dev} \simeq -2 \nu_{t,eq} S_{ij}$. Assuming a Smagorinsky-type subgrid stress $\tau_{ij}^{sgs,dev} = -2 \nu_{sgs} S_{ij}$ and minimising the Frobenius norm of the residual gives the closed form

$$\nu_{t,eq} = \nu_{sgs} - \frac{(R_{ij}^{res})^{dev} S_{ij}}{2 S_{ij} S_{ij}}, \quad (1)$$

where S_{ij} is the resolved strain-rate tensor and R_{ij}^{res} the resolved Reynolds stress tensor. By construction this is the scalar eddy viscosity that best matches the deviatoric LES stress in a least-squares sense; it deliberately discards the Reynolds-stress anisotropy in exchange for direct compatibility with a standard eddy-viscosity solver. Rather than predicting $\nu_{t,eq}$ directly, the

network is trained on the discrepancy with respect to the RANS baseline,

$$\Delta v_t = v_{t,eq} - v_{t,RANS}, \quad v_{t,ML} = v_{t,RANS} + \Delta v_{t,ML}. \quad (2)$$

2.3 Clustering and neural network

A K-means clustering algorithm [9, 8], implemented in *scikit-learn* [16], identifies the distinct flow regions of the wake, moving the prediction from a cell-wise to a region-aware basis that is more robust to mesh refinement and embedd higher generalizability. The algorithm minimises the within-cluster sum of squares, $\sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$, and the optimal number of clusters K is selected with the elbow method. Features are Min-Max normalised and projected on a cylindrical reference system centred on the rotor and aligned with the rotor axis; a subset of features is retained based on PCA and physical reasoning.

The regression model is a MLP implemented in TensorFlow-Keras [17], with a single scalar output $\Delta v_{t,ML}$. The cell-level input vector includes $\vec{U}_{cyl}$, k , $v_{t,RANS}$, the distance from the assigned cluster centroid in features space, the non-dimensional coordinates z/D and x/D , and the (one-hot encoded) cluster labels; continuous features and the target are normalised with Standard scaler algorithm. The architecture is a sequence of fully connected residual blocks (Dense layer with L2 regularisation, Layer Normalization, GELU activation and Dropout) with progressively decreasing widths, trained by minimising a weighted Huber loss with the Adam optimizer; sample weights compensate for cluster imbalance and emphasise downstream wake samples. After a first standard k - ω SST iteration, the clustering assigns a cluster to each cell and the MLP predicts $v_{t,ML}$, which is then used to advance $\vec{U}$ and p in the modified PIMPLE turbulence correction loop; the procedure is repeated every iteration. Only the eddy viscosity entering the momentum equations is replaced, while k and ω retain the standard SST transport and its boundedness treatment; the solver remained stable on all grids and inflow conditions, with no LES data used at run time.

3 CASE STUDY

The methodology is tested on the NREL-5MW [18] HAWT, modelled with the ALM with the number of nodes adapted to each grid so that $\epsilon \leq 2\Delta x$ and $\Delta b/\Delta x \leq 0.5$ [19]. A hub-height (90 m) inflow velocity $U_{ref} = 11.4$ m/s is considered as the reference (training) condition, with two further inflow velocities of 8 and 15 m/s used to assess generalization; the ABL logarithmic profile reads

$$U(z) = U_{ref} \frac{\ln((z + z_0)/z_0)}{\ln((z_{ref} + z_0)/z_0)}. \quad (3)$$

The CFD dataset was validated against literature results [20, 21] as in [9] and is omitted here for brevity. Training data are sampled along two planes through the rotor centre (one parallel and one perpendicular to the terrain) on the finest grid, yielding $\sim 180k$ cells; the far-upstream inflow region ($x/D < -2.5$) is excluded, retaining $\sim 160k$ cells split 80/20 into training and validation sets, stratified by cluster.

4 RESULTS

The clustering identifies a clear elbow at $K = 7$, separating the undisturbed flow from the inlet/ABL-affected, near-wake, mixing/breakdown and far-wake regions, consistently with the underlying physics as in previous work [9]. Trained in ~ 0.24 s, it provides the region labels used by the network.

The MLP (trained at $U_\infty = 11.4$ m/s) reaches its best checkpoint at epoch 220. In physical space the reconstructed field attains $R^2 = 0.9954$ on training and $R^2 = 0.9952$ ($RMSE = 0.2635$,

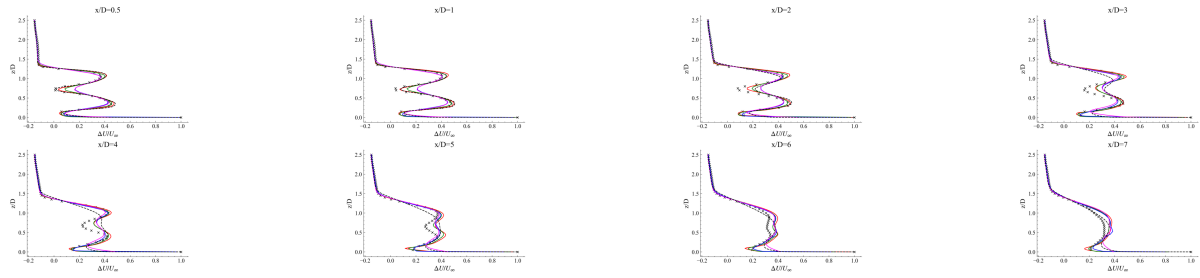


Figure 1. Velocity deficit for the 11.4 m/s inflow condition along the x -axis at different streamwise distances from the rotor, for the ML-based RANS model on grids of different resolution, compared with the standard RANS model and the LES reference.

$MAE = 0.1797$) on validation. A residual underestimation bias persists only in the inlet region, where the synthetic boundary condition introduces numerical instabilities. An equivalent network trained without clustering attains slightly worse metrics across the board ($R^2 = 0.9950$, with higher $RMSE/MAE$). This aggregate gap is small only because the metrics are dominated by the large free-stream region; the benefit of clustering is concentrated in the near-terrain region, where the clustered model is markedly more consistent with the LES reference, and in the interpretability and case- and mesh-robustness it adds.

The trained model is embedded in the RANS solver and evaluated across grids of different resolution. As shown in Fig. 1, on the fine grid (~ 15.9 M cells) the ML-based RANS model consistently outperforms the standard RANS model, especially in the upper near-wake where the standard model underestimates the deficit; a moderate underestimation persists in the lower near-wake, while the far-wake (after breakdown, from $4D$) is slightly overestimated. The most relevant result is obtained on the ultra-coarse grids: the model retains its improved near-wake prediction while also providing a much more accurate far-wake, including the post-breakdown recovery that is critical to estimate the power production of downstream turbines. This is achieved at a reduced cost: on the ~ 4.2 M cells grid the ML-based RANS model is about 1.25 times cheaper than the reference LES, making it a viable tool for large-scale wind farm simulations.

Consistently with its single-case-to-many premise, the closure transfers to the unseen 8 and 15 m/s conditions: a scalar metric based on the signed integral of the velocity-deficit difference with respect to the LES shows that, at all three inflow velocities, the standard RANS model systematically overestimates the wake deficit, whereas every RANS-ML configuration stays close to the LES reference throughout the wake. This out-of-sample transfer, rather than the single-case fit, is the central result of the study.

To assess the dynamic effects on the turbine and the associated power production, the predicted forces and the C_p/C_t coefficients are reported in Fig. 2. They show good agreement with the reference curves, with a slight underestimation partly attributable to the fact that the reference refers to an isolated turbine, whereas the ML-based RANS model operates in a turbulent ABL affected by the ground. Overall, the model reproduces not only the velocity wake field but also the aerodynamic loading and power production, a key requirement for wind farm applications.

5 CONCLUSIONS

A CFD-ML methodology was presented to build a turbulence closure that improves the fidelity of RANS simulations of HAWT wakes while keeping the computational cost compatible with wind farm applications. An LES-equivalent eddy viscosity extracted from a LES of the NREL-5MW turbine was used as target, a K-means clustering segmented the domain into

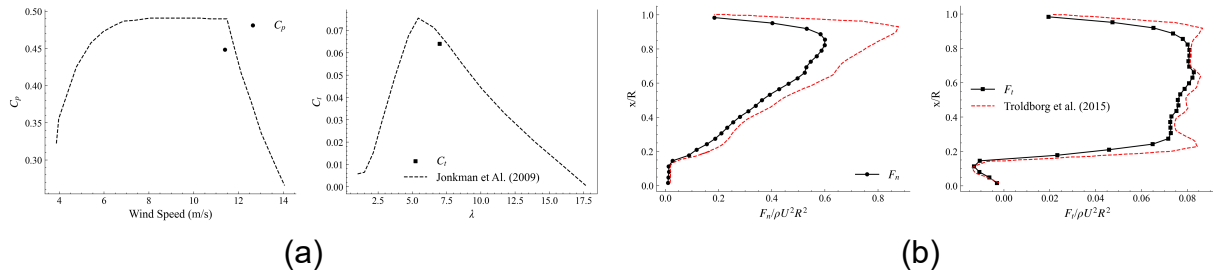


Figure 2. ML-based RANS model results: (a) C_p and C_t profiles, (b) force profiles.

seven physically meaningful flow regions, and an MLP learned the eddy-viscosity correction $\Delta \nu_t = \nu_{t,eq} - \nu_{t,RANS}$, embedded into a modified OpenFOAM PIMPLE solver. The network reproduced the LES-equivalent closure with a validation $R^2 = 0.9952$, and clustering proved decisive in the near-terrain region. Once embedded, the ML-based RANS model consistently outperformed the standard RANS model in both near- and far-wake; most notably, on ultra-coarse grids it accurately reproduced the post-breakdown far-wake recovery at a cost about 1.25 times lower than the reference LES, while also reproducing the rotor forces and C_p/C_t coefficients. By design, the closure was calibrated on a single operating condition and then transferred, without retraining, to coarser grids and to the unseen 8 and 15 m/s conditions, staying close to the LES throughout: this single-case-to-many transfer is the core contribution of the work, not a limitation of it. Future work will extend the methodology to multi-turbine configurations, different WT models, towards compact, data-driven closures suitable for real-time wind farm monitoring and control. Moreover, in future work, the pipeline will be optimised to reduce the inference time of the ML model by avoiding reliance on a Python-based pipeline, which is currently the main bottleneck of the methodology, thereby significantly further reducing computational cost and enabling real-time applications.

REFERENCES

- [1] Fernando Porté-Agel, Majid Bastankhah, and Sina Shamsoddin. Wind-turbine and wind-farm flows: A review. *Boundary-Layer Meteorology*, 174, 01 2020.
- [2] Julia Steiner, Richard Dwight, and Axelle Vire. Data-driven rans closures for wind turbine wakes under neutral conditions. *Computers & Fluids*, 233:105213, 11 2021.
- [3] J. Steiner. *Towards data-driven turbulence modeling for wind turbine wakes*. Dissertation (tu delft), Delft University of Technology, 2023.
- [4] Mehrshad Gholami Anjiraki, Christian Santoni, Samin Shapourmiandouab, Hossein Seyedzadeh, Jonathan Craig, Fotis Sotiropoulos, and Ali Khosronejad. Wind farm layout optimization using a novel machine learning approach, 09 2025.
- [5] Karthik Duraisamy, Gianluca Iaccarino, and Heng Xiao. Turbulence modeling in the age of data. *Annual Review of Fluid Mechanics*, 51, 01 2019.
- [6] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 11 2016.
- [7] R. N. King, C. Adcock, J. Annoni, and K. Dykes. Data-driven machine learning for wind plant flow modeling. *Journal of Physics: Conference Series*, 1037(7):072004, jun 2018.

- [8] Lorenzo Tieghi, Davide Cerbarano, Filippo De Girolamo, Valerio Barnabei, and Giovanni Delibra. A novel methodology for the automatic decomposition of hawt wakes with k-means clustering. *Wind Energy*, 28, 05 2025.
- [9] Andrea Carloni, Giovanni Delibra, Filippo De Girolamo, Valerio Francesco Barnabei, and Lorenzo Tieghi. Data-driven decomposition of horizontal-axis wind-turbine wakes. In *ASME Turbo Expo 2026*, Milan, Italy, 2026.
- [10] J. N. Sorensen and W. Z. Shen. Numerical modeling of wind turbine wakes. *Journal of Fluids Engineering*, 124(2):393–399, 05 2002.
- [11] Pete Bachant, Anders Goude, and Martin Wosnik. turbinesfoam. Zenodo, 04 2016.
- [12] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.
- [13] Franck Nicoud and Frédéric Ducros. Subgrid-scale stress modelling based on the square of the velocity gradient tensor. *Flow, turbulence and Combustion*, 62(3):183–200, 1999.
- [14] J. C. Kaimal, J. C. Wyngaard, Y. Izumi, and O. R. Coté. Spectral characteristics of surface-layer turbulence. *Quarterly Journal of the Royal Meteorological Society*, 98(417):563–589, 1972.
- [15] Lars Davidson. Using isotropic synthetic fluctuations as inlet boundary conditions for unsteady simulations. *Advances and Applications in Fluid Mechanics*, 1:1–35, 2007.
- [16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [17] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [18] Jason Jonkman. Definition of a 5-mw reference wind turbine for offshore system development, 2009.
- [19] Luis Martínez Tossas, Stefano Leonardi, Matthew Churchfield, and Patrick Moriarty. A comparison of actuator disk and actuator line wind turbine models and best practices for their use. 01 2012.
- [20] Lorenzo Cappugi, Alessio Castorrini, Aldo Bonfiglioli, Edmondo Minisci, and Sergio Campobasso. Machine learning-enabled prediction of wind turbine energy yield losses due to general blade leading edge erosion. *Energy Conversion and Management*, 245:114567, 10 2021.

- [21] Niels Troldborg, Frederik Zahle, Pierre-Elouan Réthoré, and Niels N. Sørensen. Comparison of wind turbine wake properties in non-sheared inflow predicted by different computational fluid dynamics rotor models. *Wind Energy*, 18(7):1239–1250, 2015.